

A practical exploration of core network protocols—including TCP, HTTP, DNS, and HTTPS—through packet-level inspection. This project demonstrates foundational skills in capturing, filtering, and analyzing network traffic, building a critical competency for cybersecurity analysis and forensic investigation.

Wireshark

Hands-on Project

Mubeen Khan

Table of Contents

Executive Summary

Requirements

Section 1: Foundation - Network Protocol Analysis with Wireshark

1.1 Lab 1: Introduction to Wireshark

1.1.1 Objective

1.1.2 Practical Exercise: First Packet Capture & HTTP Filtering

1.1.3 Results & Analysis

1.2 Lab 2: Analyzing HTTP

1.2.1 Objective

1.2.2 Practical Exercise: Inspecting HTTP Traffic

1.3 Lab 3: Analyzing TCP

1.3.1 Objective

1.3.2 Key Concepts: 3-Way Handshake, Flow Control, Reliability, Sequence/Acknowledgement Numbers

1.3.3 Practical Exercise: Tracing TCP Streams & Connection Lifecycle

1.4 Lab 4: Analyzing DHCP

1.4.1 Objective

1.4.2 Practical Exercise, Analysis and Results

Executive Summary

This portfolio documents a focused and practical exploration of defensive cybersecurity, blending academic rigor with hands-on platform training to build a foundational skillset in protecting and analysing network environments.

The work begins with a deep academic analysis of network protocols through a series of university-led Wireshark labs. This provides the critical lens through which to view network traffic, understanding the fundamental behaviour of protocols like TCP, HTTP, DNS, and TLS—a necessary skill for identifying anomalies.

This theoretical knowledge is directly applied through hands-on defensive exercises on platforms like TryHackMe and Hack The Box. These modules focused on core defensive operations, including security monitoring, digital forensics, incident response, and threat analysis within controlled environments. The practical experience gained here translates protocol-level understanding into actionable security insights.

Further specialized training from Cybrary courses has solidified concepts in network defense, threat intelligence, and security fundamentals, creating a well-rounded understanding of the cybersecurity landscape.

Together, this body of work demonstrates a committed initiative to mastering the proactive and reactive skills essential for a career in defensive security, with a proven ability to analyse systems, identify vulnerabilities, and understand the tactics of adversaries from a defender's perspective.

Requirements

This section outlines the software and configuration requirements necessary to replicate the environment used for the hands-on labs and exercises in this portfolio. All components were installed and configured on a host machine running Windows 11.

1. Virtualization Platform: Oracle VM VirtualBox

The primary lab environment was built using Oracle VM VirtualBox, a free and open-source virtualization software. This allowed for the creation of an isolated and secure Ubuntu workstation.

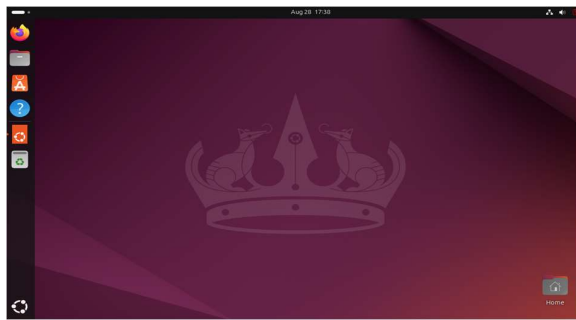
- **Version:** Oracle VM VirtualBox 7.0.xx
- **Purpose:** To host the Ubuntu virtual machine, providing hardware virtualization and network management.

2. Guest Operating System: Ubuntu Desktop

Ubuntu Linux was chosen as the guest operating system for its stability, widespread use in the cybersecurity industry, and robust command-line environment.

- **Version:** Ubuntu 24.04 LTS (Desktop)
- **Purpose:** To serve as the dedicated platform for conducting network analysis, tool installation, and practical security exercises.
- **Configuration:**
 - **Base Memory:** 4096 MB (4 GB)

- **Processors:** 3
- **Storage:** 25 GB (Dynamically Allocated VDI)
- **Network Adapter:** Adapter 1 set to **NAT** mode for secure internet access.



3. Core Analysis Tool: Wireshark

Wireshark is the industry-standard network protocol analyzer used throughout the majority of the practical labs to capture and inspect network traffic.

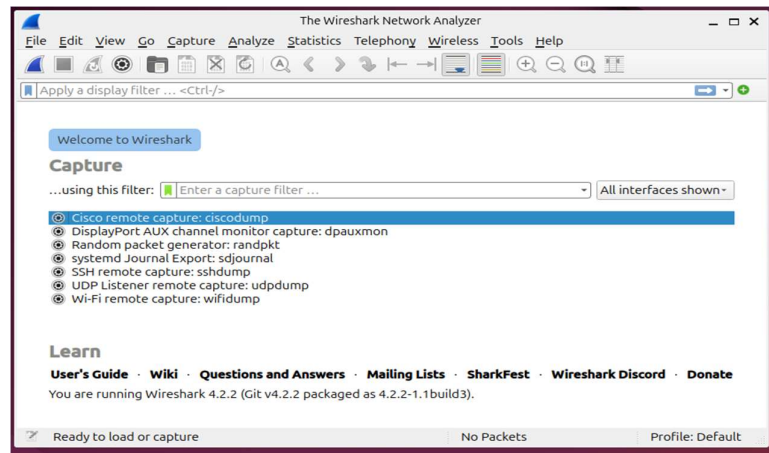
- **Version:** Wireshark 4.2.x (Installed via Ubuntu apt repository)
- **Purpose:** To perform live packet capture, conduct deep packet inspection, and analyze protocol behavior for both offensive and defensive security insights.

```
mubeen@mubeen-VirtualBox: ~
to run a command as administrator (user "root"), use "sudo <command>".
see "man sudo_root" for details.

mubeen@mubeen-VirtualBox:~$ sudo apt update
[sudo] password for mubeen:
Hit:1 http://au.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://au.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://au.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
mubeen@mubeen-VirtualBox:~$ sudo apt install wireshark
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  i965-vd-driver intel-media-va-driver libaac0 libavcodec60 libavformat60
  libavutil58 libb2-1 libbcg729-0 libbdplus0 libbluray2 libchromaprint1
  libcbjson1 libcodecs2-1.2 libdavid7 libdouble-conversion3 libgme0 libgsm1
  libhwy1t64 libigmpmm12 libjxl0.7 liblua5.2-0 libmbcrypto7t64 libmd4c0
  libminizip1t64 libnghttp3-3 libnorm1t64 libopencore-amrnb0 libopenmpt0t64
  libpcre2-16-0 libpgm-5.3-0t64 libqt6core5compat6 libqt6core6t64
```

First, I used command “**sudo apt update**” to update the list of available software packages as seen above.

Then I entered command “**sudo apt install wireshark**” to install Wireshark.



Section 1: Foundation - Network Protocol Analysis with Wireshark

1.1 Lab 1: Introduction to Wireshark

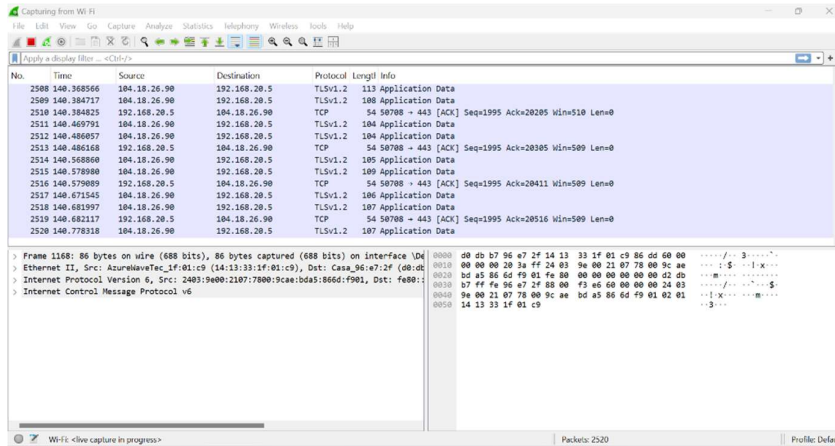
1.1.1 Objective

The purpose of this lab is to gain familiarity with Wireshark network protocol analyser. The primary goal of this lab is to understand Wireshark user interface, perform first packet capture, and apply basic filters to isolate specific HTTP traffic generated by web browsing activity.

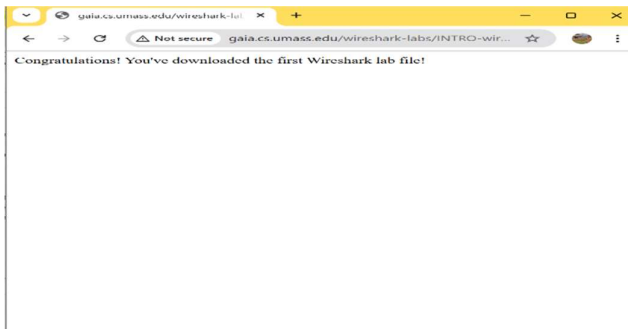
1.1.2 Practical Exercise: First Packet Capture & HTTP Filtering

The exercise involved capturing live traffic from a controlled web request and filtering the results to analyse the HTTP protocol.

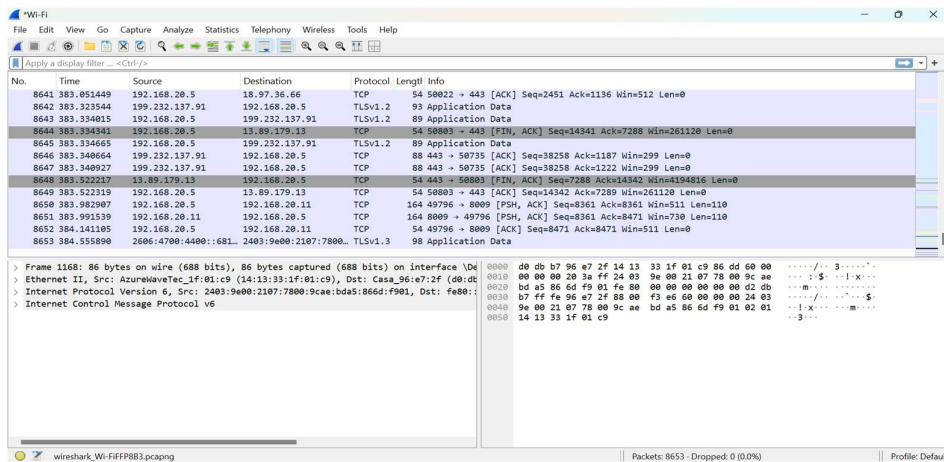
1. **Interface Selection:** The Wireshark application was launched, and the active network interface (Wi-Fi) was identified for packet capture.



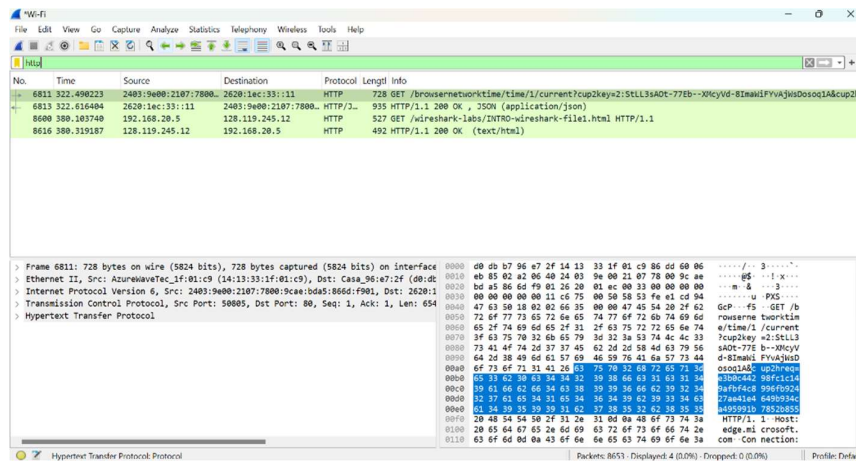
2. **Packet Capture:** Packet capture was initiated by selecting the active interface. The target URL (<http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>) was then accessed via a web browser to generate predictable HTTP traffic.



Then I stopped the capture as visible in the screenshot below:

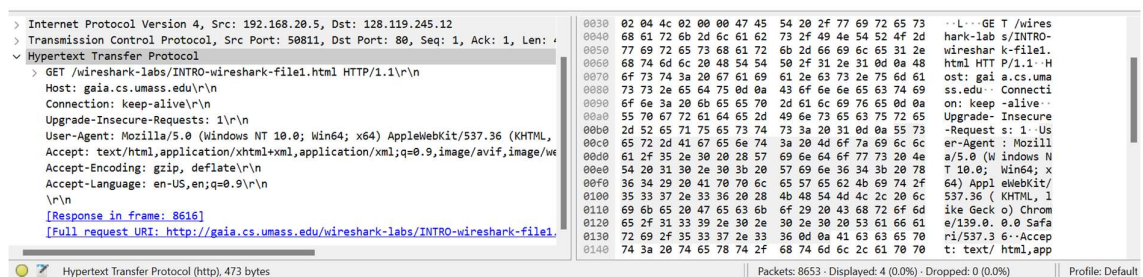


3. **HTTP Filtering:** After stopping the capture, the display filter http was applied to isolate the relevant HTTP traffic from other background protocols (e.g., DNS, TCP). This filtered view showed the HTTP GET request and the corresponding response.



1.1.3 Results & Analysis

- **Background Traffic Revealed:** Before filtering, the capture showed a significant amount of non-HTTP traffic, including DNS and TCP protocols. This highlights that even a simple web request relies on multiple underlying network services to function, which are typically hidden from the user.
- **Filtering is Essential:** Applying the http display filter instantly isolated the relevant application-layer traffic from all other network noise. This proved that mastering Wireshark filters is the most critical skill for efficient analysis.
- **Protocol Layering in Action:** Inspecting the HTTP GET request provided a clear, practical example of protocol encapsulation. The analysis showed the data being wrapped in layers: an **Ethernet** frame for the local network, an **IP** packet for routing across the internet, a **TCP** segment for reliable delivery, and finally, the **HTTP** request itself containing the GET command for the specific web page.



1.2 Lab 2: Analysing HTTP

1.2.1 Objective

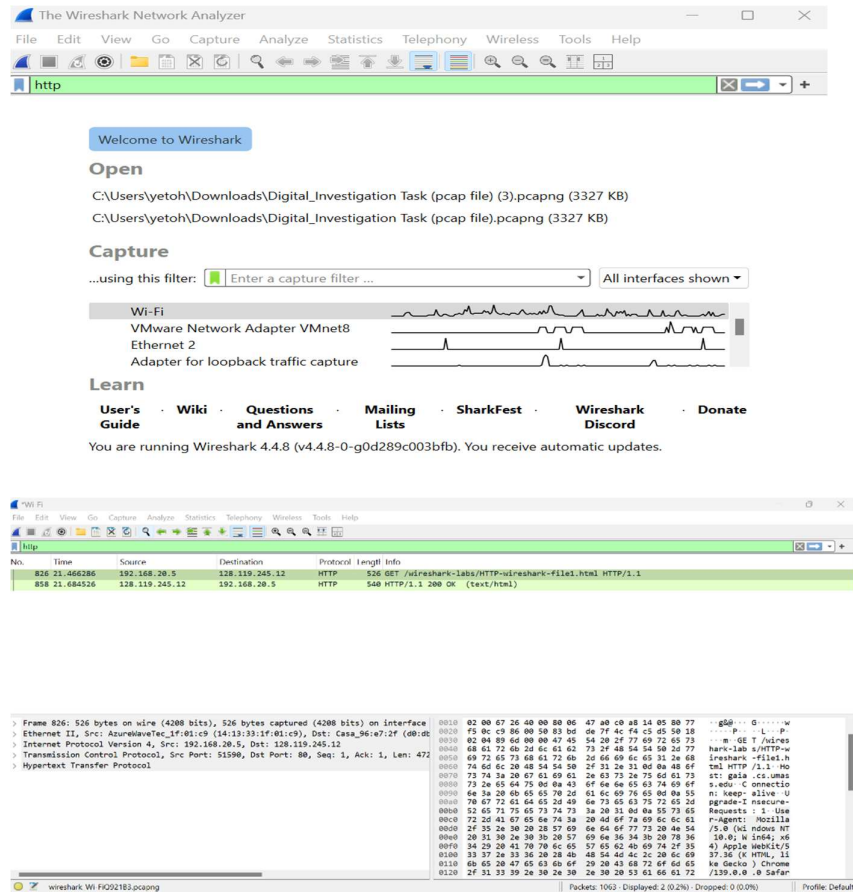
The objective of this lab was to conduct a deep dive into the Hypertext Transfer Protocol (HTTP) using Wireshark. The goal was to move beyond basic capture and analyse specific HTTP behaviours, including message formats, the effect of caching, TCP

segmentation for large files, the loading of embedded objects, and the fundamental security mechanisms of HTTP authentication.

1.2.2 Practical Exercise: Inspecting HTTP Traffic

Exercise 1:

A live packet capture was initiated in Wireshark with a pre-applied http display filter. Traffic was generated by browsing to a simple, static HTML page (<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html>).



Results and Analysis for Exercise 1:

1. HTTP Version Compatibility:

Analysis of the initial request and status lines confirmed that both the client (web browser) and the server are utilizing HTTP version 1.1

No.	Time	Source	Destination	Protocol	Length	Info
826	21.466286	192.168.20.5	129.119.245.12	HTTP	526	GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1
858	21.684526	129.119.245.12	192.168.20.5	HTTP	540	HTTP/1.1 200 OK (text/html)

<pre> > Frame 826: 526 bytes on wire (4208 bits), 526 bytes captured (4208 bits) on interface > Ethernet II, Src: AzureWaveTec_1f:01:c9 (14:13:33:1f:01:c9), Dst: Casa_96:e7:2f (08:00:0e:16:03:2f) > Internet Protocol Version 4, Src: 192.168.20.5, Dst: 129.119.245.12 > Transmission Control Protocol, Src Port: 51598, Dst Port: 80, Seq: 1, Ack: 1, Len: 0 > Hypertext Transfer Protocol > GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1\r\n Host: gaia.cs.umass.edu\r\n Connection: keep-alive\r\n Upgrade-Insecure-Requests: 1\r\n User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8) Accept-Encoding: gzip, deflate\r\n Accept-Language: en-US,en;q=0.9\r\n \r\n </pre>	<pre> 0000 d0 db b7 96 e7 2f 14 13 33 1f 01 c9 08 00 45 00 3....E- 0010 02 00 67 26 40 00 00 06 47 a0 c0 a8 54 85 80 77 ..g&...G....w 0020 f5 0c c9 05 00 50 83 bd 0e 7f 4c f4 c5 05 50 18 ...P...L...P 0030 02 04 89 6d 00 00 47 45 54 20 2f 77 69 72 65 73 ...m-GE T /wires 0040 68 61 72 65 71 68 61 72 60 2d 66 69 6c 63 31 2d 68 hark-lab s/HTTP-w 0050 74 6d 6c 20 48 54 50 2f 31 2e 31 0d 0a 48 6f 6f tnl HTTP /1.1. Ho 0060 73 74 5a 20 67 61 69 61 2a 63 73 2a 75 6d 61 73 st: gaia .cs.umass 0070 73 2a 65 64 75 0d 0a 43 6f 6a 65 63 74 69 6f 6f s.edu C connectio 0080 6e 3a 20 60 65 65 70 2d 61 6c 69 76 65 0d 0a 55 n: keep-alive-U 0090 70 67 72 61 64 65 2d 49 6a 73 65 63 75 72 65 2d ngrade-I nsecure- 0100 52 65 71 75 65 73 74 73 3a 20 31 0d 0a 55 73 65 Requests : 1 Use 0110 72 2d 41 67 65 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 r-Agent: Mozilla </pre>
--	--

2. Accepted Language:

✓ Hypertext Transfer Protocol

➤ GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1\r\n

Host: gaia.cs.umass.edu\r\n

Connection: keep-alive\r\n

Upgrade-Insecure-Requests: 1\r\n

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8)

Accept-Encoding: gzip, deflate\r\n

Accept-Language: en-US,en;q=0.9\r\n

\r\n

[\[Response in frame: 858\]](#)

[\[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html\]](http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html)

3. Source and Destination IP addresses:

The IP addresses for the communication were found in the Network Layer header. The source IP address, belonging to the client machine, was 192.168.20.5, the destination IP address, corresponding to the gaia.cs.umass.edu server, was 129.119.245.12.

```

> Frame 826: 526 bytes on wire (4208 bits), 526 bytes captured (4208 bits) on interface
> Ethernet II, Src: AzureWaveTec_1f:01:c9 (14:13:33:1f:01:c9), Dst: Casa_96:e7:2f (d0:c
✓ Internet Protocol Version 4, Src: 192.168.20.5, Dst: 128.119.245.12
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 512
    Identification: 0x6726 (26406)
> 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x47a0 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.20.5
    Destination Address: 128.119.245.12
    [Stream index: 18]
> Transmission Control Protocol, Src Port: 51590, Dst Port: 80, Seq: 1, Ack: 1, Len: 47
> Hypertext Transfer Protocol

```

4. Status code:

For the initial HTTP GET request to the simple HTML file, the server responded with a 200 OK status code. This code indicates that the request was successful, and the requested resource (the HTML file) is being returned in the response message body.

```

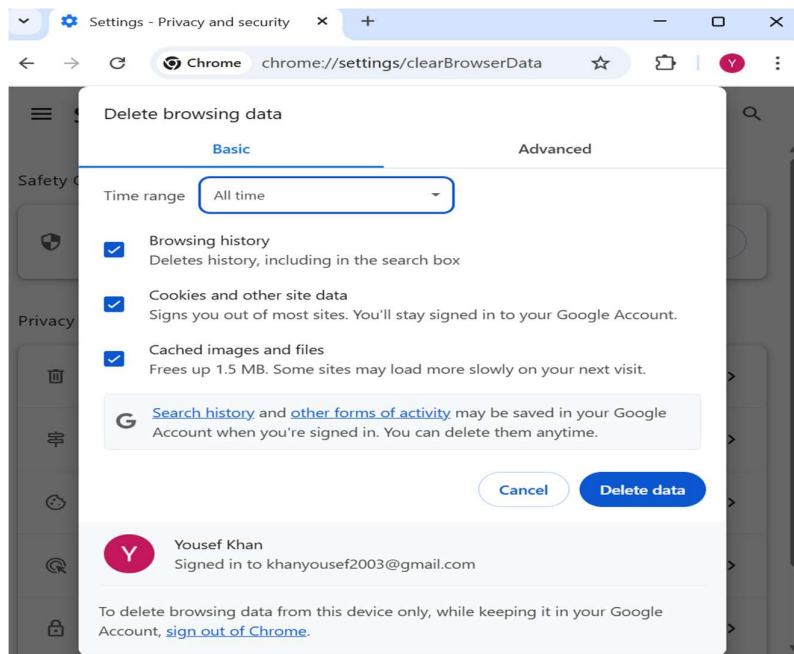
Hypertext Transfer Protocol
> HTTP/1.1 200 OK\r\n
  Date: Fri, 29 Aug 2025 09:11:02 GMT\r\n
  Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl
  Last-Modified: Fri, 29 Aug 2025 05:59:01 GMT\r\n
  ETag: "80-63d7ab7efce6a"\r\n
  Accept-Ranges: bytes\r\n
> Content-Length: 128\r\n
  Keep-Alive: timeout=5, max=100\r\n
  Connection: Keep-Alive\r\n
  Content-Type: text/html; charset=UTF-8\r\n
\r\n
[Request in frame: 826]
[Time since request: 0.218240000 seconds]
[Request URI: /wireshark-labs/HTTP-wireshark-file1.html]
[File] request ID: http://www.wireshark-labs.com/HTTP-wireshark-file1.html

```

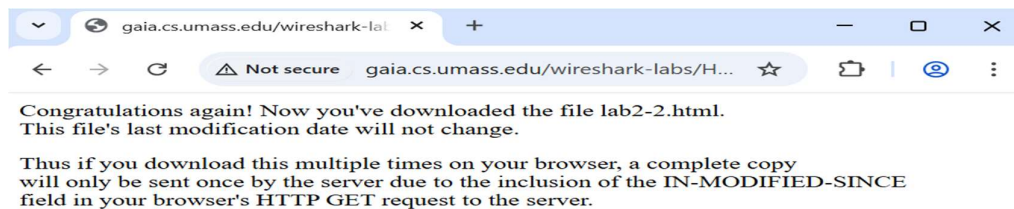
Exercise 2:

In this exercise we observe and understand how web browsers use cached copies of files and perform a "Conditional GET" to check if a file has been modified before downloading it again. This saves time and network resources.

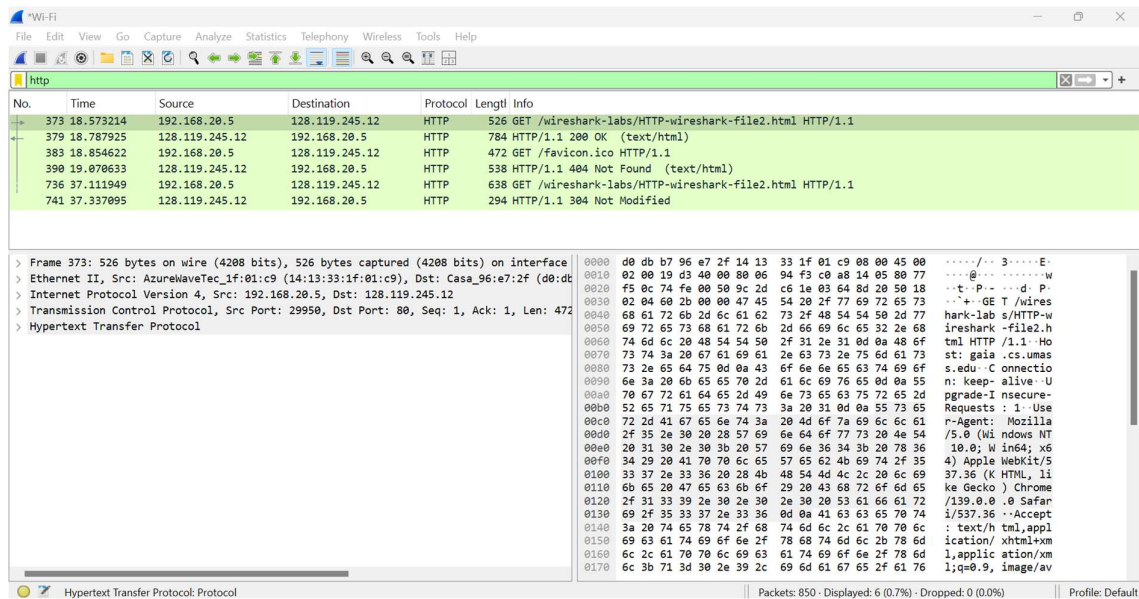
To perform this exercise, we first cleared the cache of our browser using Ctrl + Shift + Delete as seen below:



Then we start up the Wireshark packet sniffer and in our browser we entered the following URL: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>

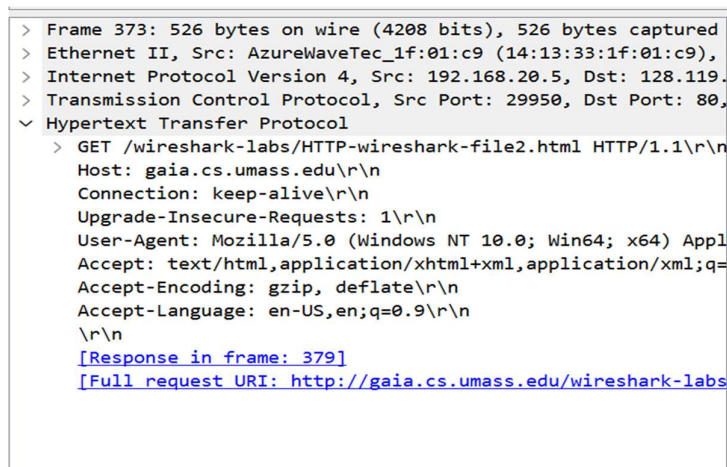


After entering the URL, we quickly entered the same URL into our browser again (or simply refreshed our browser) and then stopped the Wireshark capture. Then we entered "http" in the display-filter so that only captured HTTP messages will be displayed as observed in the screenshot below:



Results and Analysis for Exercise 2:

We analysed the responses and requests pairs. In the first HTTP GET request we cannot see IF-MODIFIED-SINCE line because the cache was cleared, and it is the first time we are asking for the file so there is no reason to check if it's modified.



We then inspect the first server response, and we found out that the server explicitly returns the contents of the file as we analysed the first response packet and found “200 OK” status code and a content-length header greater than 0 which proves the server sent the full file content.


```

> Frame 379: 784 bytes on wire (6272 bits), 784 bytes captured
> Ethernet II, Src: Casa_96:e7:2f (d0:db:b7:96:e7:2f), Dst: Azu
> Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.16
> Transmission Control Protocol, Src Port: 80, Dst Port: 29950,
✓ Hypertext Transfer Protocol
  > HTTP/1.1 200 OK\r\n
    Date: Thu, 04 Sep 2025 05:11:22 GMT\r\n
    Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.
    Last-Modified: Thu, 04 Sep 2025 05:11:01 GMT\r\n
    ETag: "173-63df2bf58db9c"\r\n
    Accept-Ranges: bytes\r\n
  > Content-Length: 371\r\n
    Keep-Alive: timeout=5, max=100\r\n
    Connection: Keep-Alive\r\n
    Content-Type: text/html; charset=UTF-8\r\n
    \r\n
    [Request in frame: 373]
    [Time since request: 0.214711000 seconds]
    [Request URI: /wireshark-labs/HTTP-wireshark-file2.html]

```

Now we inspected the second HTTP GET response and found the IF-MODIFIED-SINCE line.

```

> Frame 736: 638 bytes on wire (5104 bits), 638 bytes captured (5
> Ethernet II, Src: AzureWaveTec_1f:01:c9 (14:13:33:1f:01:c9), Ds
> Internet Protocol Version 4, Src: 192.168.20.5, Dst: 128.119.24
> Transmission Control Protocol, Src Port: 33960, Dst Port: 80, S
✓ Hypertext Transfer Protocol
  > GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n
    Host: gaia.cs.umass.edu\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleW
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-US,en;q=0.9\r\n
    If-None-Match: "173-63df2bf58db9c"\r\n
    If-Modified-Since: Thu, 04 Sep 2025 05:11:01 GMT\r\n
    \r\n
    [Response in frame: 741]
    [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/H

```

I also inspected the HTTP status code and phrase for the second response and found out “304 Not Modified” status code, which explains that the server did not send the file contents again. It just sent this status code, which tells the browser its cached copy is still valid. This saves significant bandwidth.

```

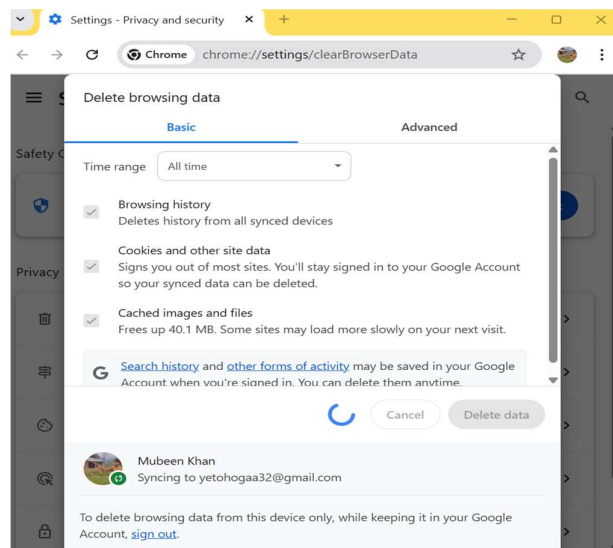
> Frame 741: 294 bytes on wire (2352 bits), 294 bytes captured (2
> Ethernet II, Src: Casa_96:e7:2f (d0:db:b7:96:e7:2f), Dst: Azure
> Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.
> Transmission Control Protocol, Src Port: 80, Dst Port: 33960, S
✓ Hypertext Transfer Protocol
  > HTTP/1.1 304 Not Modified\r\n
    Date: Thu, 04 Sep 2025 05:11:40 GMT\r\n
    Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33
    Connection: Keep-Alive\r\n
    Keep-Alive: timeout=5, max=100\r\n
    ETag: "173-63df2bf58db9c"\r\n
    \r\n
    [Request in frame: 736]
    [Time since request: 0.225146000 seconds]
    [Request URI: /wireshark-labs/HTTP-wireshark-file2.html]
    [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/H

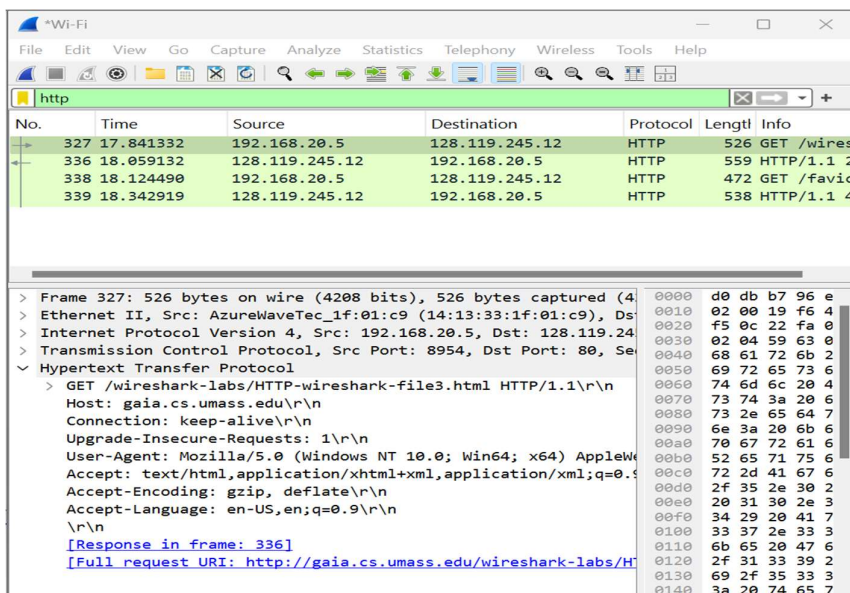
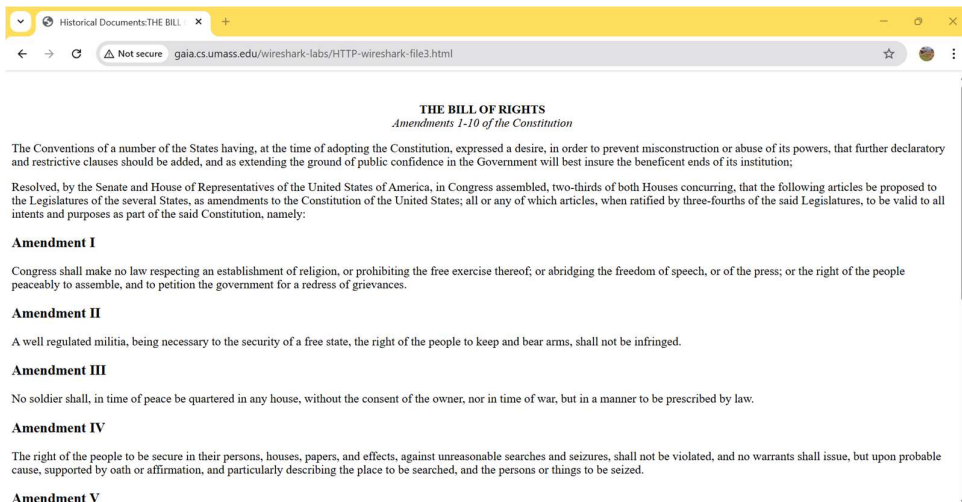
```

Exercise 3:

The objective of this exercise is to investigate how a large HTTP response is segmented and transmitted across multiple TCP packets, observing the interaction between the application protocol (HTTP) and the transport protocol (TCP).

First, we cleared the cache and then initiated a live packet capture to retrieve a lengthy HTML file from: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html>. Then we applied the display-filter, “http” to isolate the HTTP protocol messages for analysis.





Results and Analysis for Exercise 3:

HTTP Request Volume: After analysing the packet listing, I confirmed that the browser issued a single HTTP GET request to retrieve the resource. This indicates the request for the document itself is a simple, single transaction from the application's perspective.

No.	Time	Source	Destination	Protocol	Length	Info
327	17.841332	192.168.20.5	128.119.245.12	HTTP	526	GET /wireshark-labs/HTTP-wireshark-file3.html HTTP/1.1
336	18.059132	128.119.245.12	192.168.20.5	HTTP	559	HTTP/1.1 200 OK (text/html)
338	18.124490	192.168.20.5	128.119.245.12	HTTP	472	GET /favicon.ico HTTP/1.1
339	18.342919	128.119.245.12	192.168.20.5	HTTP	538	HTTP/1.1 404 Not Found (text/html)

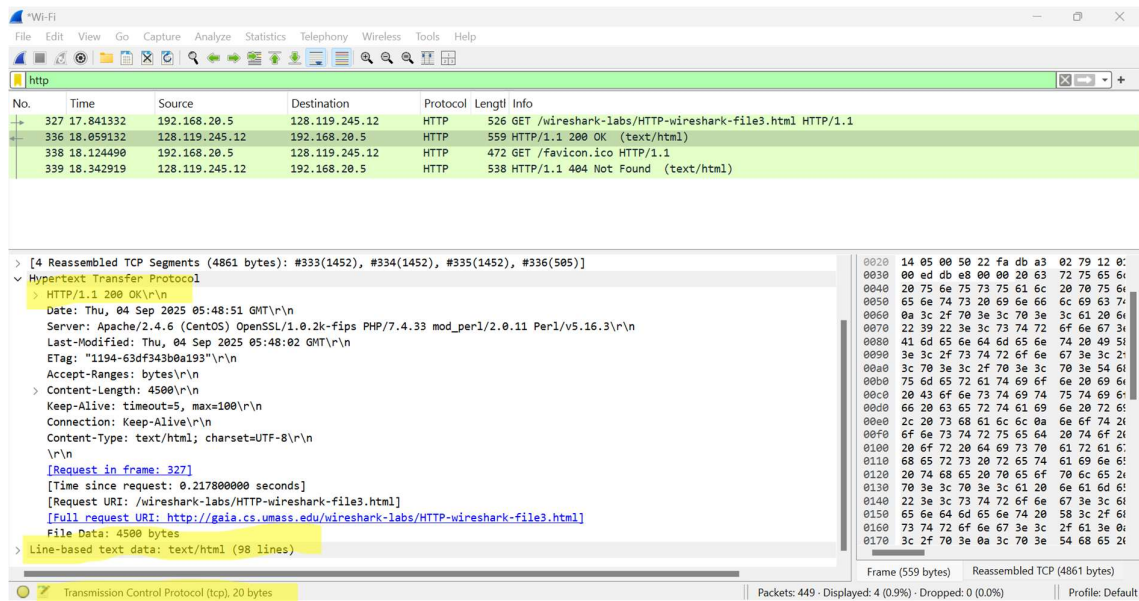
> Frame 327: 526 bytes on wire (4208 bits), 526 bytes captured (4208 bits) on interface \Device\NPF_{13DF348B-E1A4-4238-BD14-9A4D75}	0000	d0 db b7 96 e7 2f 14 13 33 1f 01 c5
> Ethernet II, Src: AzureWaveTec_1f:01:c9 (14:13:33:1f:01:c9), Dst: Casa_96:e7:2f (d0:db:b7:96:e7:2f)	0010	02 00 19 f6 40 00 80 06 94 d0 c0 a1
> Internet Protocol Version 4, Src: 192.168.20.5, Dst: 128.119.245.12	0020	f5 0c 22 fa 00 50 12 01 6c 82 db a1
> Transmission Control Protocol, Src Port: 8954, Dst Port: 80, Seq: 1, Ack: 1, Len: 472	0030	02 04 59 63 00 00 47 45 54 20 2f 7f
> Hypertext Transfer Protocol	0040	68 61 72 6b 2d 6c 61 62 73 2f 48 54
> GET /wireshark-labs/HTTP-wireshark-file3.html HTTP/1.1\r\n	0050	69 72 65 73 68 61 72 6b 2d 66 69 6a
Host: gaia.cs.umass.edu\r\n	0060	74 6d 6c 20 48 54 54 50 2f 31 2e 31
Connection: keep-alive\r\n	0070	73 74 3a 20 67 61 69 61 2e 63 73 2a
Upgrade-Insecure-Requests: 1\r\n	0080	73 2e 65 64 75 0d 0a 43 6f 6e 6e 6f
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36\r\n	0090	6e 3a 20 6b 65 65 70 2d 61 6c 69 7f
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n	00a0	70 67 72 61 64 65 2d 49 6e 73 65 6f
Accept-Encoding: gzip, deflate\r\n	00b0	52 65 71 75 65 73 74 73 3a 20 31 0a
Accept-Language: en-US,en;q=0.9\r\n	00c0	72 2d 41 67 65 6e 74 3a 20 4d 6f 7f
\r\n	00d0	2f 35 2e 30 20 28 57 69 6e 64 6f 7f
[Response in frame: 336]	00e0	20 31 30 2e 30 3b 20 57 69 6e 36 3a
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html]	00f0	34 29 20 41 70 70 6c 65 57 65 62 41
	0100	33 37 2e 33 36 20 28 4b 48 54 4d 4c
	0110	6b 65 20 47 65 63 6b 6f 29 20 43 6d
	0120	2f 31 33 39 2e 30 2e 30 2e 30 20 5f
	0130	69 2f 35 33 37 2e 33 36 0d 0a 41 6f
	0140	3a 20 74 65 78 74 2f 68 74 6d 6c 2c
	0150	69 63 61 74 69 6f 6e 2f 78 68 74 6c
	0160	6c 2c 61 70 70 6c 69 63 61 74 69 6f

TCP Segmentation of HTTP Response: The data for single HTTP response was contained within a single TCP segment. This is evident by a single HTTP/1.1 200 OK packet in the capture, with no following “continuation” frames.

Successful Response Status: The server responded to the initial GET request with a “200 OK” status code, indicating that the entire document was available and would be delivered.

No.	Time	Source	Destination	Protocol	Length	Info
327	17.841332	192.168.20.5	128.119.245.12	HTTP	526	GET /wireshark-labs/HTTP-wireshark-file3.html HTTP/1.1
336	18.059132	128.119.245.12	192.168.20.5	HTTP	559	HTTP/1.1 200 OK (text/html)
338	18.124490	192.168.20.5	128.119.245.12	HTTP	472	GET /favicon.ico HTTP/1.1
339	18.342919	128.119.245.12	192.168.20.5	HTTP	538	HTTP/1.1 404 Not Found (text/html)

Lastly, no TCP-induced “Continuation” was necessary, as the entire response was delivered in one segment. Therefore, there are no additional HTTP status lines, as the single “200 OK” status line successfully governed the entire response.



Exercise 4:

The objective of this Exercise 4 is to analyse the behaviour of a web browser when loading a page containing embedded resources (images) stored on external servers. The focus was on determining the number of requests generated and the strategy (serial or parallel) used to retrieve these objects.

We started by clearing the browser cache, followed by initiating packet capture. The browser was directed to a URL: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html>. This URL contain an HTML file that references two external images. Then we stop the capture and display-filter “http” was applied to isolate the HTTP protocol messages for analysis.

Delete browsing data

Basic

Advanced

Time range
Last hour

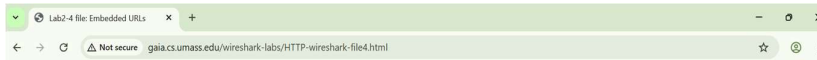
☒ Browsing history
Deletes history, including in the search box

☒ Cookies and other site data
Signs you out of most sites

☒ Cached images and files
Frees up less than 15.2 MB. Some sites may load more slowly on your next visit.

Cancel

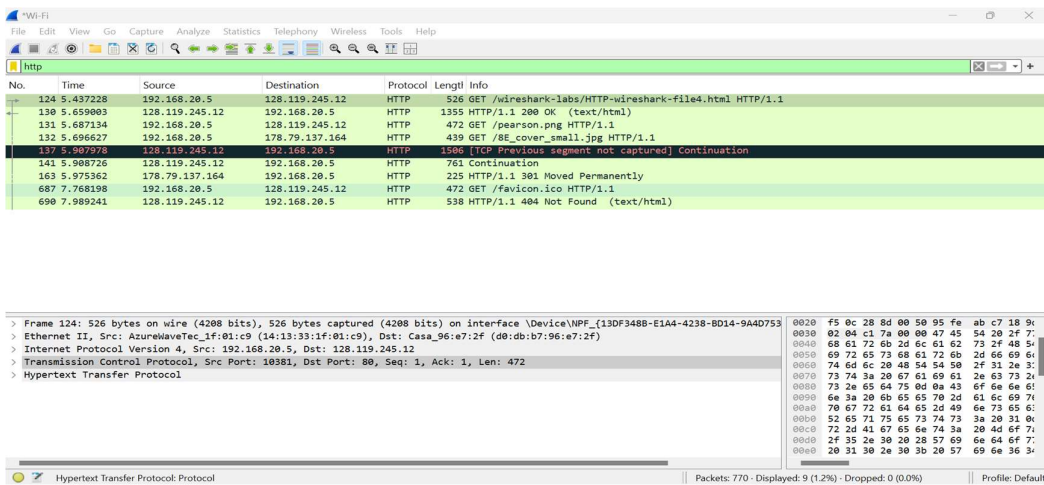
Delete data



This little HTML file is being served by gaia.cs.umass.edu. It contains two embedded images. The image above, also served from the gaia.cs.umass.edu web site, is the logo of our publisher, Pearson. The image of our 8th edition book cover below is stored at, and served from, a WWW server kurose.cslash.net in France:



And while we have your attention, you might want to take time to check out the available open resources for this book at http://gaia.cs.umass.edu/kurose_ross.



Results and Analysis for Exercise 4

The capture revealed the multi-step process a browser undertakes to fully render a webpage.

The analysis reveals that the browser sent four primary **HTTP GET requests** to three different internet addresses to construct a webpage:

- 1) **GET for the Base HTML Page:** The first request was for the main HTML file, sent to the primary server at 128.119.245.12 (gaia.cs.umass.edu).
- 2) **GET for the First Image (pearson.png):** The browser issued a request for the first embedded image. This was also sent to 128.119.245.12, indicating the image was stored on the same server.
- 3) **GET for the second image (book cover):** The browser simultaneously issued a request for the second embedded image. This request was sent to a different server at IP address 178.79.137.164.
- 4) **GET for Favicon:** An additional, automatic request for favicon.ico was sent to 128.119.245.12 which failed with a “**404 Not Found**”.

No.	Time	Source	Destination	Protocol	Length	Info
124	5.437228	192.168.20.5	128.119.245.12	HTTP	526	GET /wireshark-labs/HTTP-wireshark-file4.html HTTP/1.1
130	5.659003	128.119.245.12	192.168.20.5	HTTP	1355	HTTP/1.1 200 OK (text/html)
131	5.687134	192.168.20.5	128.119.245.12	HTTP	472	GET /pearson.png HTTP/1.1
132	5.696627	192.168.20.5	178.79.137.164	HTTP	439	GET /BE_cover_small.jpg HTTP/1.1
137	5.907978	128.119.245.12	192.168.20.5	HTTP	1506	[TCP Previous segment not captured] Continuation
141	5.908726	128.119.245.12	192.168.20.5	HTTP	761	Continuation
163	5.975362	178.79.137.164	192.168.20.5	HTTP	225	HTTP/1.1 301 Moved Permanently
687	7.768198	192.168.20.5	128.119.245.12	HTTP	472	GET /favicon.ico HTTP/1.1
690	7.989241	128.119.245.12	192.168.20.5	HTTP	538	HTTP/1.1 404 Not Found (text/html)

From my analysis, I found out that the browser downloaded two images in parallel. After looking at the timing of the HTTP GET requests, as shown in the **Time** column:

- The GET request for /pearson.png (packet 131) occurred at timestamp 5.687134.
- The GET request for /BE_cover_small.jpg (packet 132) occurred at timestamp 5.696627.

There is a 9.5 millisecond difference of delay between these two requests which shows a parallel behaviour. The browser optimized performance by opening multiple connections to retrieve the independent images simultaneously, rather than waiting for one to finish before starting the next (serially).

No.	Time	Source	Destination	Protocol	Length	Info
124	5.437228	192.168.20.5	128.119.245.12	HTTP	526	GET /wireshark-labs/HTTP-wireshark-file4.html HTTP/1.1
130	5.659003	128.119.245.12	192.168.20.5	HTTP	1355	HTTP/1.1 200 OK (text/html)
131	5.687134	192.168.20.5	128.119.245.12	HTTP	472	GET /pearson.png HTTP/1.1
132	5.696627	192.168.20.5	178.79.137.164	HTTP	439	GET /BE_cover_small.jpg HTTP/1.1
137	5.907978	128.119.245.12	192.168.20.5	HTTP	1506	[TCP Previous segment not captured] Continuation
141	5.908726	128.119.245.12	192.168.20.5	HTTP	761	Continuation
163	5.975362	178.79.137.164	192.168.20.5	HTTP	225	HTTP/1.1 301 Moved Permanently
687	7.768198	192.168.20.5	128.119.245.12	HTTP	472	GET /favicon.ico HTTP/1.1
690	7.989241	128.119.245.12	192.168.20.5	HTTP	538	HTTP/1.1 404 Not Found (text/html)

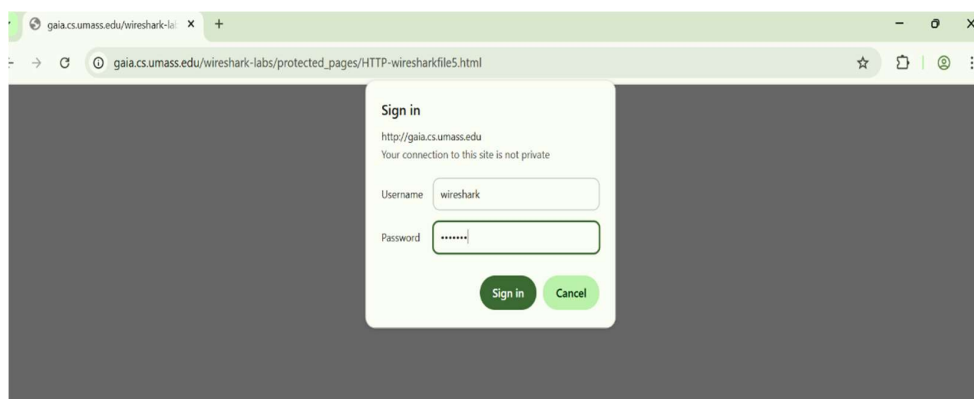
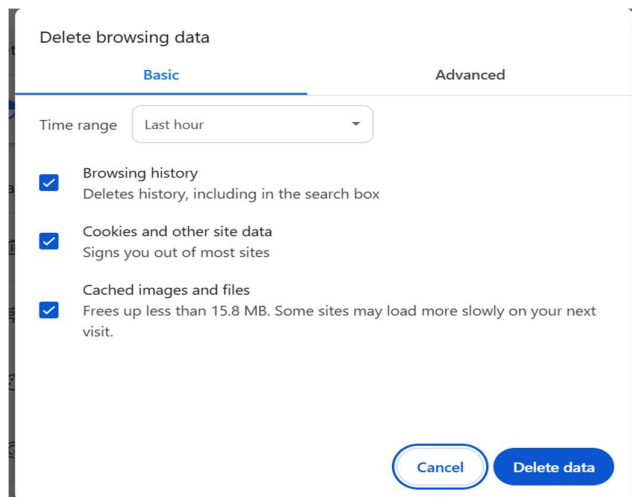
Exercise 5:

The objective of this exercise is to examine a sequence of HTTP messages exchanged for accessing a password-protected website and to examine security properties of the HTTP Basic Authentication mechanism.

We started by clearing the browser cache, followed by initiating a new packet capture. The browser was then directed to a password-protected URL:

http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html.

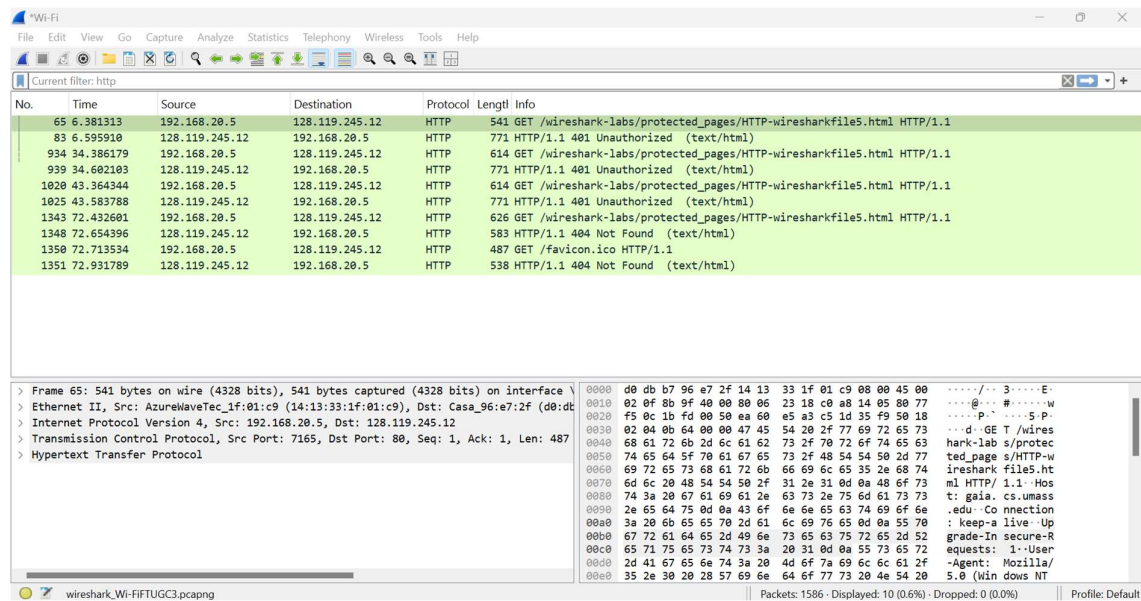
The credentials (username: wireshark-students, password: network) were entered into the browser's authentication pop-up dialog. We then stopped the packet capture and used Wireshark display-filter "http" to isolate the HTTP protocol messages for analysis.



Results and Analysis for Exercise 5

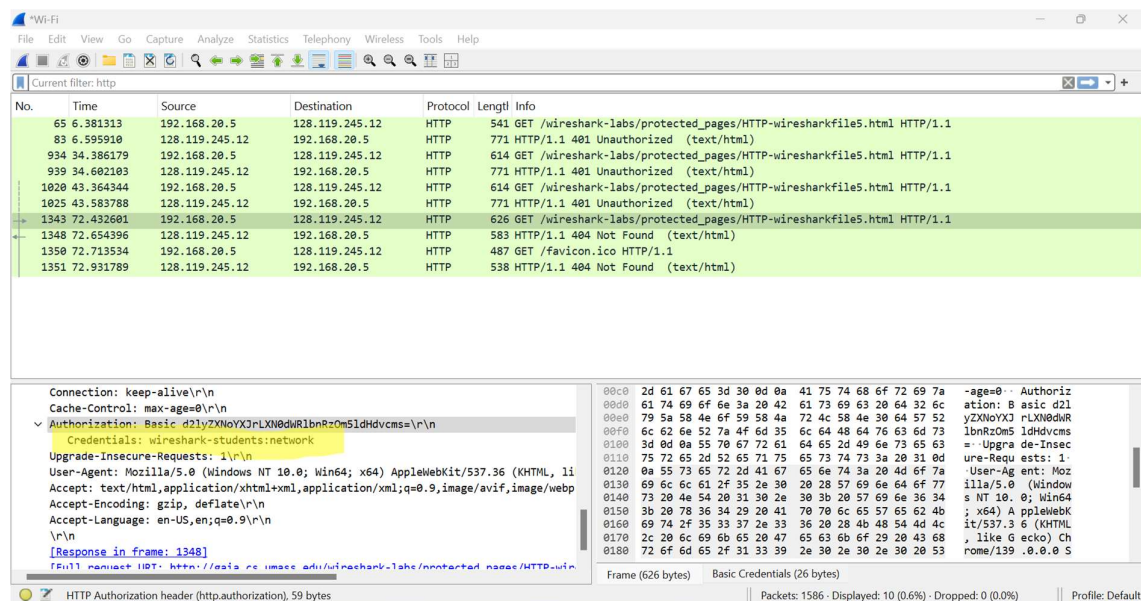
I found the server's response to initial HTTP GET message from my browser was **"401 Unauthorized"**. This was a standard challenge response from a server protecting a resource with Basic Authentication. It tells the client to provide valid credentials to access the resource.

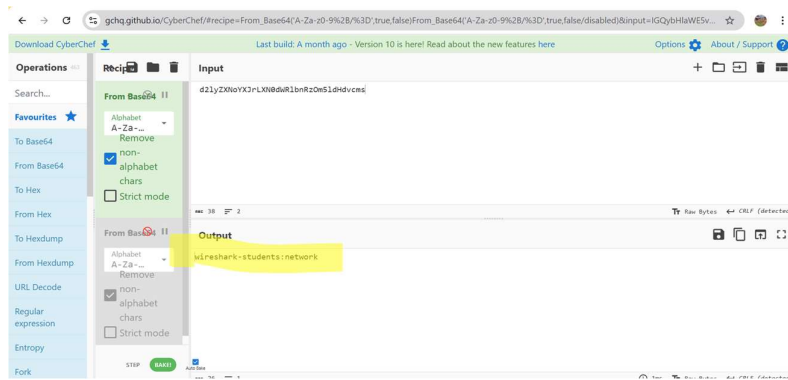
After I enter correct credentials, the GET request include a new header field: Authorization: Basic <credentials>. This header is the browsers attempt to answer the server's challenge by providing the encoded username and password.



Key Security Analysis

The authorization header, even with correct credentials, is only Base64-encoded, not encrypted. Anyone capturing this traffic can easily decode it to reveal credentials or just inspect the packet to find out the credentials.





This demonstrates why Basic Authentication over plain HTTP is extremely insecure. It also highlights the critical importance of **HTTPS (HTTP over TLS/SSL)**. HTTPS would encrypt the entire conversation, including the Authorization header, preventing this eavesdropping.

1.3 Lab 3: Analyzing TCP

1.3.1 Objective

The primary objective of this Lab is to investigate the behaviour of Transmission Control Protocol in detail using Wireshark. By analysing a trace of the TCP segments sent and received in transferring a 150KB file. The purpose of this lab is to provide a practical understanding of TCP's core mechanisms that ensure reliable, ordered, and efficient data delivery across a network.

1.3.2 Key Concepts: 3-Way Handshake, Flow Control, Reliability, Sequence/Acknowledgement Numbers

TCP is a reliable, connection-oriented transport protocol that ensures ordered, error-checked delivery of data between applications; used for web browsing, email, and file transfers.

3-Way Handshake: A logical connection must be established between the client and server through a three-step process:

- Client to Server **SYN**: Client initiates connection by sending **SYN** packet to the server, the packet contains random initial sequence number.
- Server to Client **SYN-ACK**: The server responds to the client with **SYN-ACK** packet which adds the initial sequence numbers randomly chosen by the server.
- Client to Server **ACK**: Client sends ACK packet to acknowledge the reception of SYN-ACK packet. Thereby the connection is established, and data transfer can begin.

Flow Control: It prevents a fast sender from overwhelming a slow receiver, managed using the Window Size field in the TCP header.

Congestion Control: Prevents a sender from overwhelming the network, TCP uses **Congestion Window** to limit the amount of data it can inject into the network before receiving an ACK.

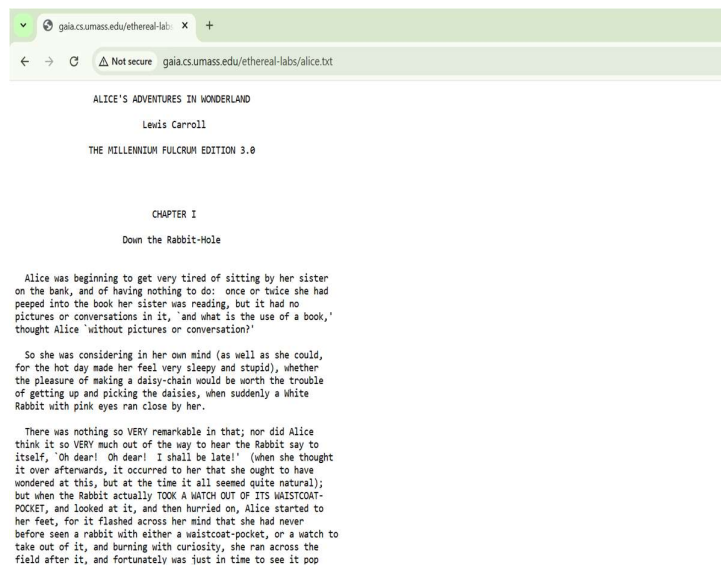
TCP Stream: Represents the bidirectional flow of data within a single connection between two endpoints.

1.3.3 Practical Exercise: Tracing TCP Streams & Connection Lifecycle

Exercise 1

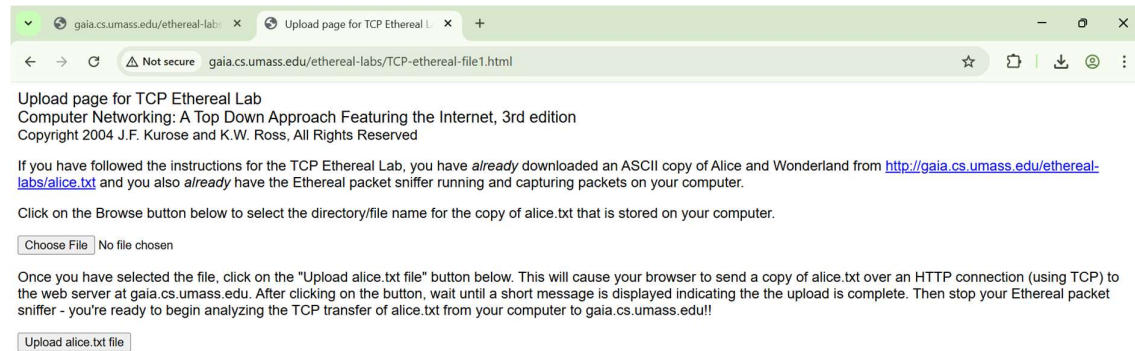
In this exercise we will be capturing a bulk TCP transfer from my computer to a remote server, we'll use Wireshark to obtain a packet trace of the TCP transfer of a file from my computer to a remote server. This will be done by accessing a **Web Page** that will allow us to enter the name of a file stored on my computer (which in this exercise will contain the ASCII text of Alice in Wonderland) and then transfer the file to a Web server using the **HTTP POST** method. The reason for using **POST** method instead of **GET** method is that we'd like to transfer large amount of data from my computer to another computer. During this time our Wireshark will be running to obtain the trace of the TCP segments sent and received from your computer.

We begin by starting up our web browser and going to the <http://gaia.cs.umass.edu/ethereal-labs/alice.txt> where we retrieve an ASCII copy of Alice in Wonderland, which will be stored on our computer.

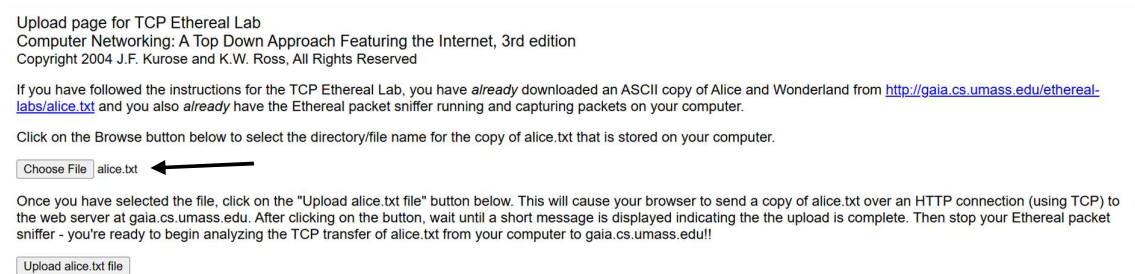


Then I right-clicked and clicked “save-As” and saved the entire text of the book onto my desktop as Alice.txt.

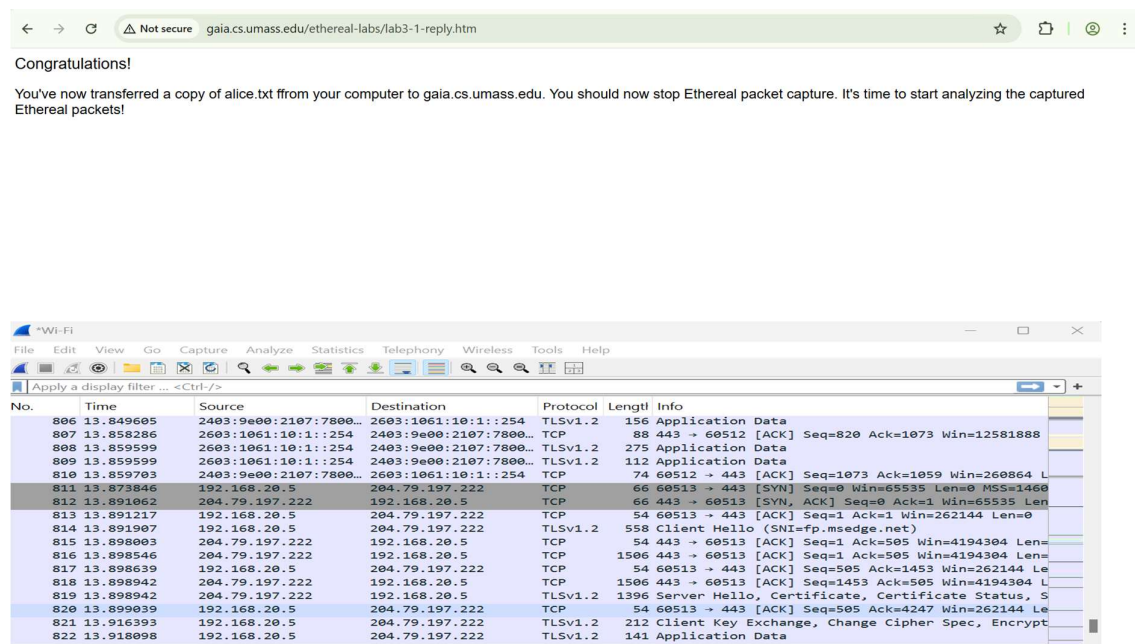
Next, I went to: to <http://gaia.cs.umass.edu/ethereal-labs/TCP-ethereal-file1.html> as instructed by the exercise:



We will then click on “Choose File” and upload the Alice.txt file. However, we will only press the “upload alice.txt” file only when we start our Wireshark capture.



I have now started my Wireshark capture and then clicked the “Upload alice.txt” file.



This is the end of **Exercise 1**.

Exercise 2

This Exercise is the continuation of Exercise 1. We will be performing initial, high-level inspection of the capture TCP traffic. This involves identifying the key components of the TCP connection between the client and the server. The goal is to establish foundational understanding of the communication endpoints and the structure of the TCP stream before diving into a more detailed analysis of the protocol's behaviour. We will be familiarizing ourselves with three-way handshake.

Results & Analysis

First, we find out the IP address and TCP port number of my computer for transferring the file. I used the filter **"http.request.method == "POST"**. My **source port** is: **"60502"** and **IP address** is **"192.168.20.5"**.

The image shows a Wireshark packet capture window with the filter `http.request.method == "POST"`. The packet list shows a single packet (No. 273) at time 6.167748, from source 192.168.20.5 to destination 128.119.245.12, protocol HTTP, length 663 bytes. The packet details pane shows the following structure:

- Frame 273: 663 bytes on wire (5304 bits), 663 bytes captured (5304 bits) on interface
- Ethernet II, Src: AzureWaveTec_1f:01:c9 (14:13:33:1f:01:c9), Dst: Casa_96:e7:2f (d0:db:00:00:00:00)
- Internet Protocol Version 4, Src: 192.168.20.5, Dst: 128.119.245.12
- Transmission Control Protocol, Src Port: 60502, Dst Port: 80, Seq: 152417, Ack: 1, Len: 663
- HTTP, Method: POST, Path: /ethereal-labs/lab3-1-reply.htm, Version: 1.1, Content-Type: text/plain

The packet bytes pane shows the raw data of the packet, including the Ethernet II header, IP header, and the HTTP POST request body.

I further analysed the IP address of `gaia.cs.umass.edu` and the port number it is sending and receiving TCP segment for this connection.

Destination IP address: 128.119.245.12

Destination Post: 80

1.4 Lab 4: Analysing DHCP

1.4.1 Objective

In this Lab, we will be investigating the behaviour of Dynamic Host Configuration Protocol (DHCP) by capturing and analysing its network traffic. DHCP is critical for automatically assigning IP address and network configuration parameters to clients on a network.

The key goals were to:

- **Observer the DHCP Four-Way Handshake:** Capture the sequence of DHCP messages (Discover, Offer, Request, ACK) to understand how a client dynamically obtains an IP address from a server.
- **Analyse Protocol Mechanics:** Decoding the structure of DHCP packets to examine key fields such as transaction IDs, offered IP addresses, and lease times.
- **Understand the underlying Transport:** Identify the transport layer protocol and port number used for DHCP communications.
- **Investigate Initial Client State:** Explore how IP datagrams are constructed during the DHCP process before the client has a confirmed IP address.
- **Examine Network Integration:** Identify the role of accompanying protocols like ARP during the DHCP exchange and understand concepts like DHCP relay agents.

1.4.2 Practical Exercise, Analysis and Results

We will start our lab using the following steps:

1. Preparation

Open command prompt as administrator and type “ipconfig /all”

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.22631.5909]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>ipconfig /all

Windows IP Configuration

Host Name . . . . . : MK
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : home

Unknown adapter OpenVPN Wintun:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Wintun Userspace Tunnel
Physical Address. . . . . :
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes

Ethernet adapter Ethernet 2:

Connection-specific DNS Suffix . :
Description . . . . . : VirtualBox Host-Only Ethernet Adapter
Physical Address. . . . . : 0A-00-27-00-00-16
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::ce4b:a613:686f:b6b5%22(Preferred)
IPv4 Address. . . . . : 192.168.56.1(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
DHCPv6 IAID . . . . . : 688521255
DHCPv6 Client DUID. . . . . : 00-01-00-01-2A-44-10-F2-14-13-33-1F-01-C9
NetBIOS over Tcpip. . . . . : Enabled

Unknown adapter Local Area Connection 2:
```

2. Release IP Address

Type “ipconfig /release”, our IP address will become 0.0.0.0

```
C:\Windows\System32>ipconfig /release

Windows IP Configuration

No operation can be performed on Local Area Connection 2 while it has its media disconnected.
No operation can be performed on Local Area Connection 3 while it has its media disconnected.
No operation can be performed on Local Area Connection* 1 while it has its media disconnected.
No operation can be performed on Local Area Connection* 2 while it has its media disconnected.

Unknown adapter OpenVPN Wintun:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix . :
    Link-local IPv6 Address . . . . . : fe80::ce4b:a613:686f:b6b5%22
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Unknown adapter Local Area Connection 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

Unknown adapter Local Area Connection 3:

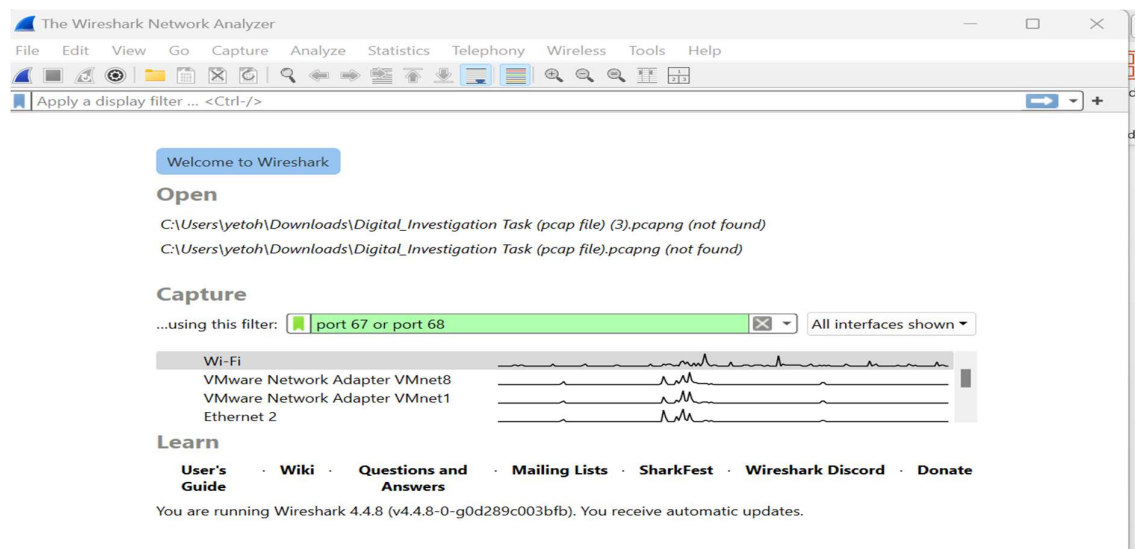
    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

Unknown adapter OpenVPN Data Channel Offload:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :
```

3. Start Wireshark Capture

We then opened Wireshark, selected active network interface (Wi-Fi) and set the capture filter: **“port 67 or port 68”** to capture only DHCP traffic and then started our capture.



4. Renew IP Address

Now we went back to command prompt and type **“ipconfig /renew”** and watched DHCP 4-way Handshake on Wireshark.


```
Administrator: Command Prompt
C:\Windows\System32>ipconfig /renew

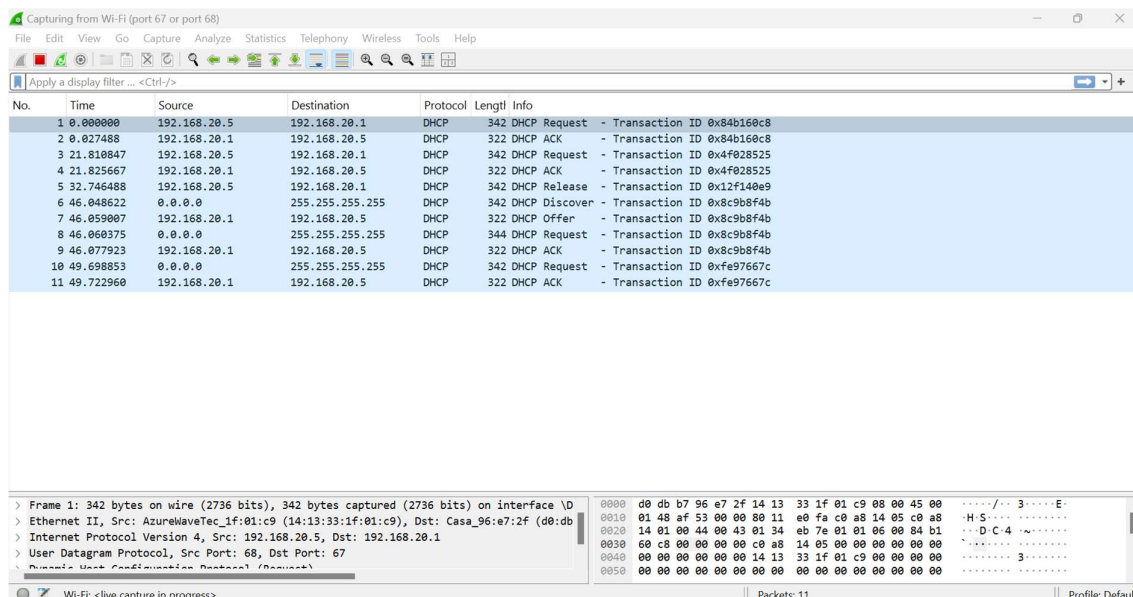
Windows IP Configuration

No operation can be performed on OpenVPN Wintun while it has its media disconnected.
No operation can be performed on Local Area Connection 2 while it has its media disconnected.
No operation can be performed on Local Area Connection 3 while it has its media disconnected.
No operation can be performed on OpenVPN Data Channel Offload while it has its media disconnected.
No operation can be performed on Local Area Connection* 1 while it has its media disconnected.
No operation can be performed on Local Area Connection* 2 while it has its media disconnected.

Unknown adapter OpenVPN Wintun:
```

5. Continue the sequence

After the renewal is complete we type “**ipconfig /renew**” again and then **ipconfig /release** and finally **ipconfig /renew** once more before stopping Wireshark Capture.



Analysis

First Sequence (Packet 1-2): We started Wireshark after computer already had an IP (192.168.20.5), so we can only see renewal conversation (Request and ACK).

Second Sequence (Pack 6-9): This is the complete DHCP 4-way Handshake after we did **ipconfig /release** and **ipconfig /renew**:

Packet 6: DHCP Discover

Packet 7: DHCP Offer

Packet 8: DHCP Request

Packet 9: DHCP ACK

Further Analysis

- **The DHCP Messages are sent over UDP**

```
> Frame 2: 322 bytes on wire (2576 bits), 322 bytes captured (2576 bits) on interface \D
> Ethernet II, Src: Casa_96:e7:2f (d0:db:b7:96:e7:2f), Dst: AzureWaveTec_1f:01:c9 (14:13:3
✓ Internet Protocol Version 4, Src: 192.168.20.1, Dst: 192.168.20.5
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 308
    Identification: 0x0000 (0)
> 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0xd062 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.20.1
```

- **Link Layer Host Address**

```
> Frame 2: 322 bytes on wire (2576 bits), 322 bytes captured (2576 bits) on interface \Dev:
✓ Ethernet II, Src: Casa_96:e7:2f (d0:db:b7:96:e7:2f), Dst: AzureWaveTec_1f:01:c9 (14:13:3
    > Destination: AzureWaveTec_1f:01:c9 (14:13:33:1f:01:c9)
    > Source: Casa_96:e7:2f (d0:db:b7:96:e7:2f)
        Type: IPv4 (0x0800)
        [Stream index: 0]
> Internet Protocol Version 4, Src: 192.168.20.1, Dst: 192.168.20.5
> User Datagram Protocol, Src Port: 67, Dst Port: 68
> Dynamic Host Configuration Protocol (ACK)
```

Conclusion

This series of practical labs provided a comprehensive, hands-on exploration of fundamental network protocols using Wireshark as the primary analysis tool. Through guided experimentation, I progressed from basic packet capture operations to analyzing the intricate behaviors of core application and transport layer protocols.

The key skills demonstrated and solidified throughout this project include:

- **Proficient Protocol Analysis:** Gained the ability to capture, filter, and dissect live network traffic to understand the structure and sequence of HTTP, TCP, and DHCP communications.

- **TCP/IP Suite Deep Dive:** Developed a practical understanding of the TCP connection lifecycle—including the 3-way handshake, data transfer mechanics with sequence/acknowledgment numbers, and connection termination—and contrasted it with the connectionless nature of DHCP over UDP.
- **DHCP Process Examination:** Mapped the client-server interaction of the DHCP DORA (Discover, Offer, Request, Acknowledge) process, identifying how IP addresses are dynamically assigned and the significance of key fields like Transaction-ID.
- **Practical Troubleshooting Acumen:** Cultivated the essential skill of using a protocol analyzer to verify normal protocol behavior and diagnose potential network issues based on observed traffic patterns.

This project successfully transitioned theoretical networking concepts into tangible, observed phenomena, establishing a strong foundation for further work in network administration, security, and application development.